ReBAC V3.1 — Zanzibar Relations & Permission Composition

> ***Branch:*** `ReBAC-V3.1` | ***Builds on:*** *V3.0 Identity Graph*

The Problem V3.0 Left Unsolved

In V3.0, every permission is individually defined with full redundancy:

```
permission view = doctor_of OR assigned_nurse OR department_head
permission edit = doctor_of OR department_head
permission discharge = doctor_of OR department_head
```

`doctor_of OR department_head` is duplicated across `edit` and `discharge`. Any change requires updating both. At scale with hundreds of resources, this becomes a maintenance crisis.

The solution: Permissions should be *composable* — one permission can reference another.

```
permission medical_staff = doctor_of OR assigned_nurse
permission view = medical_staff OR department_head
permission edit = medical_staff                        ← reuse!
```

This is Zanzibar's **Computed Userset** — the most elegant idea in the system.

V3.1 Architecture

1. Computed Usersets

A permission that references **another permission within the same resource** is a computed userset:

```
resource patient {
    relation doctor_of : user
    relation assigned_nurse : user
    relation department_head : user

    # Computed userset — medical_staff is a derived permission set
    permission medical_staff = doctor_of OR assigned_nurse

    # Computed userset reference — evaluates medical_staff, not a
relation
    permission view = medical_staff OR department_head
}
```

Evaluation:

```
checkPermission(user, patient, "view")
→ Evaluate "view" = medical_staff OR department_head
→ Evaluate "medical_staff" = doctor_of OR assigned_nurse
→ Check doctor_of → true → ALLOW
```

2. Dot Notation Tuple-to-Userset

```
permission view = parent.view
```

Replaces the verbose arrow notation:

```
permission view = contains -> view   (V2.x style)
permission view = parent.view        (V3.1 style)
```

Both mean: "the set of users who have `view` permission on my parent". The dot notation is closer to Zanzibar's own specification syntax.

3. Permission Dependency Graph & Cycle Detection

Computed usersets can create dependency cycles:

```
permission view = edit
```

```
permission edit = view   ← CYCLE!
```

The compiler must detect this at **compile time** using topological sort on the permission dependency graph:

```
DFS with 3-color marking:
  - UNVISITED (white) → start DFS
  - VISITING (gray) → currently on DFS stack
  - VISITED (black) → fully processed

If we reach a VISITING node → cycle detected → compile error
```

This is exactly the same algorithm used in TypeScript's type checker to detect circular type references.

4. SubjectResolver — Pre-computation Optimization

The `SubjectResolver` resolves the membership graph **once** before any rule evaluation:

```
// One BFS expansion before rule evaluation
const subjectSet = await
SubjectResolver.resolveSubjectSet(userId);

// Pass to RuleEngine — no repeated BFS during rule evaluation
RuleEngine.evaluate(subjectSet, resource, permission);
```

This avoids re-querying the identity graph on every rule branch evaluation.

Critical Review

✅ What V3.1 Gets Right

1. **Computed usersets are semantically correct.** This is faithful to Zanzibar Section 2.3 — "userset rewrites" allow permissions to be defined in terms of other permissions.
2. **Topological sort cycle detection** is the correct algorithm for permission dependency checking. TypeScript, Python import system, and GNU Make all use the same approach.
3. **SubjectResolver pre-computation** is a correct optimization — matching SpiceDB's approach of resolving identity graph expansion before resource-graph evaluation.

4. **Dot notation** (`parent.view`) is more readable than arrow notation (`contains -> view`) and closer to Zanzibar's own syntax.

⚠️ What V3.1 Is Still Missing

Missing: Zanzibar `userset rewrite rules` — full model

Zanzibar defines three types of userset operations:

1. `this` — the direct relation on this object
2. `computed_userset` — a permission computed from other permissions
3. `tuple_to_userset` — traverse to a related object, apply a permission there

V3.1 implements (2) and (3) but not the full compositional model of (1) combined with (2) and (3) as Zanzibar defines them.

Missing: Wildcard/Public relations

```
relation viewer: user:*   ← "all users" — Zanzibar's wildcard userset
```

Missing: Exclusion operator

```
permission view = viewer - banned_user   ← OpenFGA/Zanzibar exclusion
```

OpenFGA and SpiceDB both support set difference (`-`) for explicit permission exclusion. V3.1 only has AND/OR/NOT.

Missing: Typed subject sets

```
relation viewer: user | group#member   ← OpenFGA typed relation
```

❌ Incorrect Assumption: Computed Usersets Are Pure

The RuleEngine evaluates computed usersets through recursive `checkPermission` calls. This means:

- Deep permission chains create deep call stacks

- No memoization across permission boundaries (e.g., `medical_staff` is not memoized between `view` and `edit` evaluations)

The memo key should include the `permission` name so `medical_staff` results are cached across calls.

Summary

V3.1 brings the ADSL compiler semantically close to Google Zanzibar by implementing Computed Usersets, Tuple-to-Userset dot notation, and compile-time cycle detection. The design is faithful to Zanzibar's core concepts. The main gaps are the missing exclusion operator (`-`), wildcard usersets (`user:*`), and typed subject set relations — all of which are present in production Zanzibar implementations like SpiceDB and OpenFGA.